



Connectify

Production Peer-to-Peer Video Conferencing Platform
Complete Codebase Architectural Analysis & Interview Guide

React 19

WebRTC P2P

Socket.IO 4.8

Node.js & Express 5

MongoDB & Mongoose

Author: Sujan
Poojary

• **Platform:**
Connectify v2.0

• **Scope:** Full Frontend, Backend, Signaling
& Data Flow

STEP 1: Project Folder Structure

The Connectify repository is structured as a full-stack monorepo featuring an Express + Node.js ES-Module backend and a Create React App (CRA) React 19 frontend:

```

Connectify/
├─ README.md
├─ .gitignore
├─ backend/
│   └─ package.json
bcrypt, jwt)
├─ package-lock.json
├─ src/
│   └─ app.js
express pipeline
│   └─ controllers/
│       └─ socketManager.js
state, rooms)
│   └─ user.controller.js
getUserHistory)
│   └─ middleware/
│       └─ auth.middleware.js
routes
│   └─ models/
│       └─ meeting.model.js
│       └─ user.model.js
│   └─ routes/
│       └─ users.routes.js
└─ frontend/
    └─ README.md
    └─ package.json
react-router-dom)
    └─ package-lock.json
    └─ public/
        └─ index.html
        └─ manifest.json
        └─ favicon.ico
        └─ logo192.png / logo512.png
        └─ mobile.png / background.png
        └─ robots.txt
    └─ src/
        └─ index.js
        └─ index.css
resets
    └─ App.js
    └─ App.css
styles
    └─ App.test.js
    └─ background.png
    └─ contexts/
        └─ AuthContext.jsx
Axios client
    └─ utils/
        └─ withAuth.jsx
    └─ pages/
        └─ landing.jsx
visual)
    └─ authentication.jsx
    └─ home.jsx
recent meetings)
    └─ history.jsx
functionality
    └─ VideoMeet.jsx
chat, settings
    └─ styles/
        └─ landing.module.css

```

Top-level repository overview

Ignores node_modules, .env files, builds, logs

Node.js + Express + Socket.IO Server

Backend dependencies (express 5, socket.io, mongoose,

HTTP & WebSocket entrypoint, Mongoose DB connection,

Socket.IO event handler (WebRTC signaling, chat, media

REST controllers (login, register, addToHistory,

JWT verification middleware protecting user activity

Mongoose Schema for meeting history records

Mongoose Schema for registered users

Express router mapping /api/v1/users endpoints

React 19 Single Page Application

React scripts documentation

Frontend dependencies (MUI, socket.io-client, axios,

Public static assets

HTML entrypoint with viewport & fonts

PWA web app manifest

React 19 root bootstrap (`createRoot`) with StrictMode

Global CSS design tokens, typography, CSS variables,

React Router configuration & AuthProvider wrapper

Global app shell styles, auth page styles, dashboard

Default CRA test suite

Background graphic asset

React Context: authentication state, JWT decoding,

Higher-Order Component (HOC) guarding protected routes

Public SaaS landing page (Hero, features, interactive

Login / Registration dual-tab authentication card

Protected user dashboard (instant meeting, join,

Meeting logs view with search & copy code

Real-time WebRTC room, dynamic grid, media toggles,

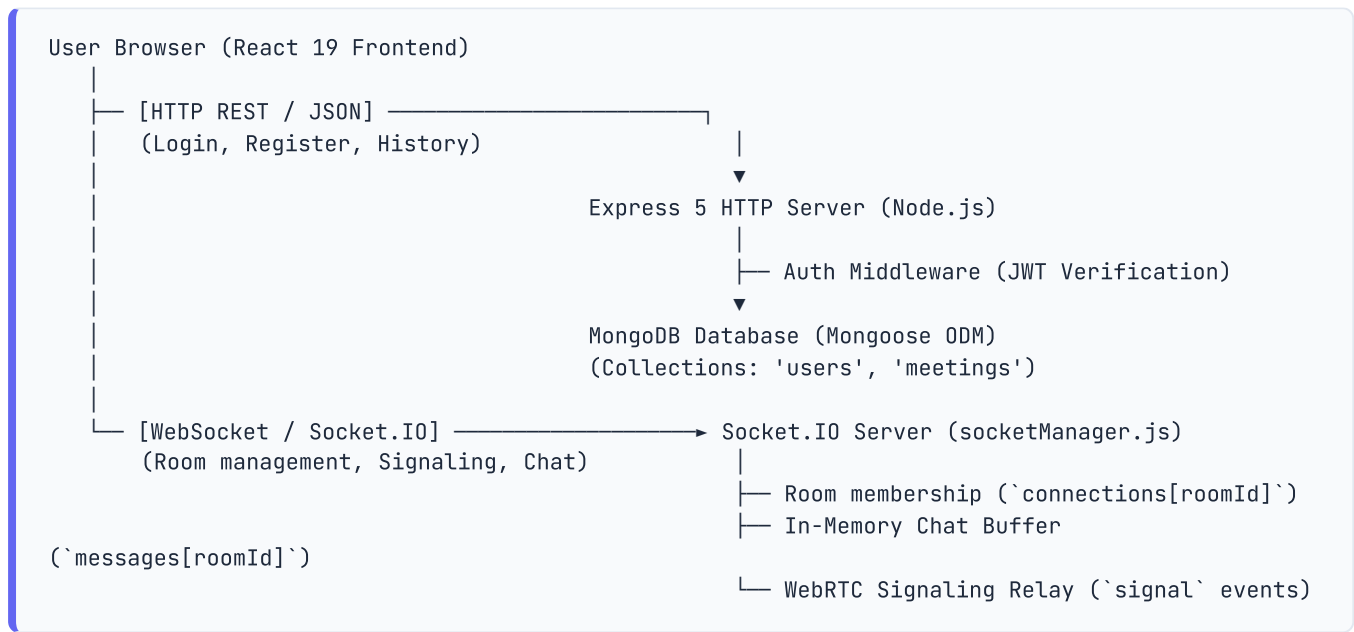
CSS Modules for LandingPage

```
└─ videoComponent.module.css # CSS Modules for VideoMeet (100dvh viewport, dynamic  
grid)
```

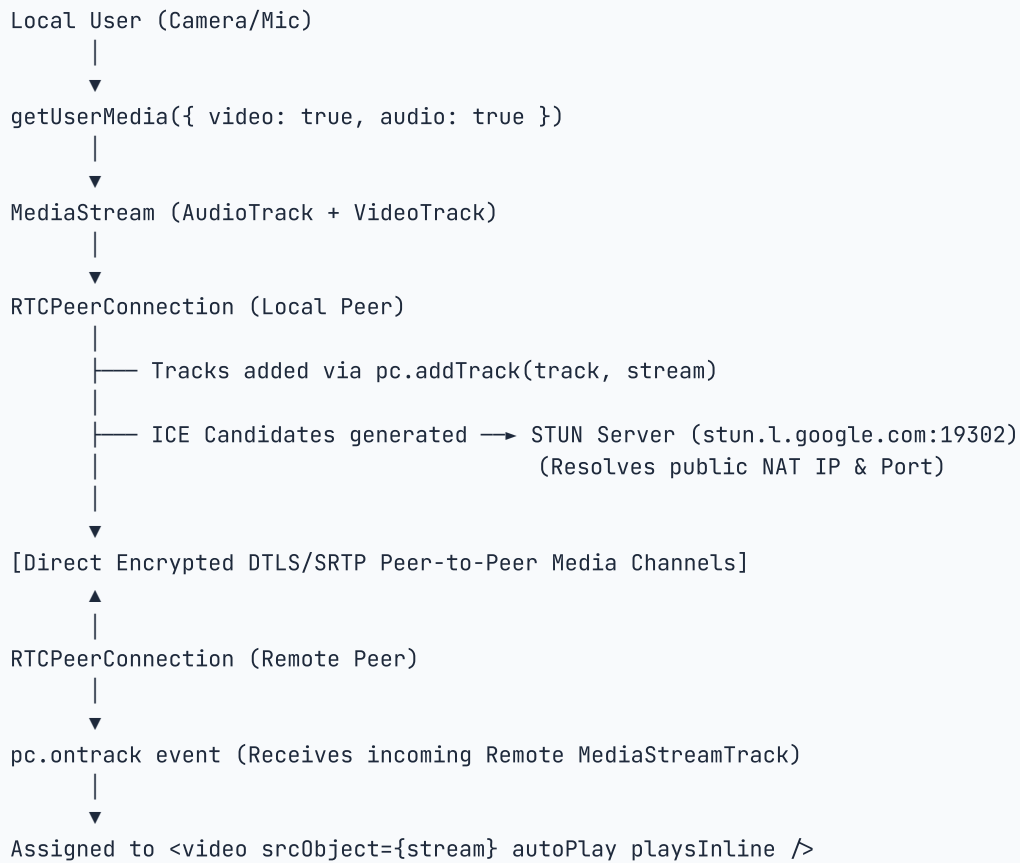
STEP 2: Architecture Overview

Connectify adopts a hybrid architectural topology: **Client-Server architecture for Authentication, Persistence, and WebRTC Signaling**, coupled with a **Mesh Peer-to-Peer (P2P) WebRTC architecture for Video and Audio Media Transport**.

1. High-Level System Architecture



2. WebRTC Video/Audio Call Media Architecture



3. Signaling Architecture (Offer / Answer / ICE Exchange)



STEP 3: Frontend Files Analysis

FILE: frontend/src/index.js

Purpose: Application entrypoint initializing the DOM root rendering pipeline.

Used by: Webpack / Create React App build pipeline.

Imports: `React`, `ReactDOM/client`, `./index.css`, `./App`.

What happens internally: Uses React 19's `ReactDOM.createRoot()` targeting `document.getElementById('root')` and renders `<App />` inside `<React.StrictMode>`. `StrictMode` intentionally mounts components twice in development mode to catch missing effect cleanups.

If removed: The application fails to boot; blank page rendered.

Interview Concept: React 18/19 Concurrent Root API vs legacy `ReactDOM.render`; React `StrictMode` double invocation behavior on WebSockets and media streams.

FILE: frontend/src/App.js

Purpose: Top-level application router and Context hierarchy definition.

Used by: `src/index.js`.

Imports: `BrowserRouter`, `Routes`, `Route` from `react-router-dom` (v7); **Pages:** `LandingPage`, `Authentication`, `VideoMeetComponent`, `HomeComponent`, `History`; `AuthProvider`.

Architecture & Hierarchy:

```
BrowserRouter
├── AuthProvider (Provides auth token, user profile, login/register methods)
│   └── Routes
│       ├── Route / - LandingPage
│       ├── Route /auth - Authentication
│       ├── Route /home - HomeComponent (Wrapped in withAuth HOC)
│       ├── Route /history - History (Guarded by withAuth logic via context)
│       └── Route /:url - VideoMeetComponent (Dynamic room URL)
```

If removed: Client-side navigation breaks; no routes or auth context provided.

FILE: `frontend/src/contexts/AuthContext.jsx`

Purpose: Centralized authentication state management, JWT persistence, and Axios API client configuration.

Used by: `App.js`, `landing.jsx`, `authentication.jsx`, `home.jsx`, `history.jsx`, `VideoMeet.jsx`.

Key Internals:

- Creates custom Axios instance: `baseUrl = (process.env.REACT_APP_API_URL || "http://localhost:8000") + "/api/v1/users"`.
- **Request Interceptor:** Automatically retrieves `localStorage.getItem("token")` and attaches `config.headers.Authorization = `Bearer ${token}`` to every outgoing HTTP call.
- `decodeToken(token)`: Parses base64url payload `atob(token.split('.')[1])` on initial mount to restore session without requiring an extra network roundtrip.
- `cleanName(val)`: Sanitizer that strips whitespace and prevents placeholder strings like `"unknown"`, `"null"`, or `"undefined"` from contaminating state.

Exposed Methods: `handleLogin`, `handleRegister`, `getHistoryOfUser`, `addToUserHistory`, `userData`, `setUserData`.

If removed: Complete auth layer collapse; users cannot log in, register, or query meeting history.

FILE: `frontend/src/utils/withAuth.jsx`

Purpose: Higher-Order Component (HOC) that enforces route guarding on protected views.

Used by: `home.jsx` (`export default withAuth(HomeComponent)`).

What happens internally: On mount, runs a `useEffect` checking `localStorage.getItem("token")`. If absent, redirects immediately via `useNavigate()` to `/auth`; otherwise renders `<WrappedComponent {...props} />`.

Interview Concept: HOC pattern vs modern React Router v6/v7 Protected Route layout components (`<Outlet />`).

FILE: `frontend/src/pages/VideoMeet.jsx` (The Core Real-Time Engine)

Purpose: Orchestrates the entire WebRTC peer-to-peer audio/video calling mesh, Socket.IO signaling, real-time in-call chat, screen sharing, device selection, and meeting UI.

Key Subcomponents:

- `MeetingTimer` (memoized): Self-contained 1-second interval timer. Isolated so timer ticks never cause video elements or remote participant tiles to re-render.
- `LocalVideo` (memoized): Renders local user camera/screen track. Has `defaultMuted=true` and `muted=true` to prevent acoustic feedback/echo loops.
- `RemoteVideo` (memoized): Renders remote peer video stream with avatar fallbacks and floating live reaction badges.

State & Ref Management:

- `peerConnectionsRef.current`: Key-value map of `socketId → RTCPeerConnection`. Stored in a ref to avoid re-rendering the component when connection states change.
- `iceCandidateQueuesRef.current`: Candidate buffer preventing race conditions when ICE candidates arrive before SDP `setRemoteDescription()` completes.
- `seenMsgIds.current`: `Set<string>` preventing duplicate chat message inserts.
- `explicitLeaveRef.current`: Flag shielding WebRTC tracks from accidental teardown during React 19 StrictMode double-mounting.

If removed: The core conferencing feature of Connectify ceases to exist.

STEP 4: Backend Files Analysis

FILE: `backend/src/app.js`

Purpose: Primary server bootstrapper combining Express HTTP server with Socket.IO and Mongoose database connection.

Used by: Node.js runtime (`npm run dev` / `npm start`).

Key Internals:

- Imports `dotenv/config` at line 1 to load `.env` before any dependencies execute.
- Creates native HTTP server with `node:http.createServer(app)`.
- Attaches Socket.IO via `connectToSocket(server)`.
- Configures CORS with wildcard origin and JSON body parser with `40kb` payload limit to defend against large payload DoS attacks.
- Connects to MongoDB via `mongoose.connect(process.env.MONGODB_URI)`.
- Handles `EADDRINUSE` port conflicts gracefully.

FILE: backend/src/controllers/socketManager.js

Purpose: State manager and signaling relay for all active WebSocket connections.

In-Memory Stores:

- `connections = {}`: `roomId → [socketId, ...]`
- `messages = {}`: `roomId → [{ id, sender, senderName, message, senderSocketId, timestamp }]` (capped at 100 messages per room)
- `socketToRoom = {}`: `socketId → roomId`
- `socketNames = {}`: `socketId → displayName`
- `mediaStates = {}`: `socketId → { videoEnabled, audioEnabled }`

Supported Socket Events:

- `join-call`: Adds user to room, broadcasts `user-joined` to all peers, and replays message history to the newly joined peer.
- `signal`: Blindly routes WebRTC offers, answers, and ICE candidates between specific socket IDs: `io.to(toId).emit("signal", socket.id, message)`.
- `chat-message`: Generates unique `msgId`, appends to 100-item circular buffer, and broadcasts to room.
- `video-state` / `audio-state`: Updates track state and broadcasts state changes to room.
- `reaction`: Broadcasts emoji reactions to room peers.
- `disconnect`: Cleans up room membership, notifies remaining peers via `user-left`, and deletes room state if empty.

FILE: backend/src/controllers/user.controller.js

Purpose: Controller handling user registration, credential verification, JWT token issuance, and meeting history operations.

Endpoints Implemented:

- `register`: Validates inputs (name 2-50 chars, username alphanumeric 3-30 chars, password ≥ 8 chars), hashes password with `bcrypt.hash(password, 10)`, and saves to MongoDB.
- `login`: Verifies user existence and compares password hash via `bcrypt.compare()`. Issues signed JWT with 7-day expiration: `jwt.sign({ _id, username, name }, JWT_SECRET, { expiresIn: '7d' })`.
- `getUserHistory`: Queries `Meeting.find({ user_id: req.user.username }).sort({ date: -1 })`.
- `addToHistory`: Inserts a new `Meeting` record linked to `req.user.username`.

FILE: `backend/src/middleware/auth.middleware.js`

Purpose: Protects private API routes by verifying the presence and signature of incoming JWT tokens.

How it works: Extracts `Authorization: Bearer <token>` header. Calls `jwt.verify(token, process.env.JWT_SECRET)`. If valid, attaches decoded payload to `req.user` and invokes `next()`. If token is invalid or absent, halts execution with HTTP 401 Unauthorized.

STEP 5: Database Analysis

Connectify uses **MongoDB** with **Mongoose ODM**.

Database Schema Models

Model	Field	Type	Constraints	Purpose
User user.model.js	name	String	required: true	User's display full name
	username	String	required: true, unique: true	Unique account login handle
	password	String	required: true	Bcrypt salted password hash
Meeting meeting.model.js	user_id	String	Indexed	References user.username
	meetingCode	String	required: true	Alphanumeric room identifier
	date	Date	default: Date.now, required: true	Timestamp of session creation/join

Database Optimization: In backend/src/models/meeting.model.js line 12, a compound index is defined: `meetingSchema.index({ user_id: 1, date: -1 });` This directly optimizes the history query `Meeting.find({ user_id: req.user.username }).sort({ date: -1 })` into an indexed prefix scan, ensuring lightning-fast dashboard loading.

STEP 6: Authentication Architecture

Connectify implements a stateless JSON Web Token (JWT) authentication flow with client-side localStorage persistence:



STEP 7: WebRTC Deep Dive (Real Implementation)

Connectify implements a **Full Mesh WebRTC** architecture where every participant maintains an active peer-to-peer connection with every other participant in the room.

Core WebRTC Primitives in Connectify

- `navigator.mediaDevices.getUserMedia()` : Invoked once on mount in `VideoMeet.jsx` (line 424) requesting `{ video: true, audio: true }`. Returns local `MediaStream`.
- `RTCPeerConnection(ICE_SERVERS)` : Instantiated dynamically per remote peer using Google's public STUN servers (`stun:stun.l.google.com:19302`).
- `addTrack(track, stream)` : Attaches local audio and video tracks to the peer connection so they stream to the remote peer.
- `ontrack` : Event listener on `RTCPeerConnection`. Triggered when the remote peer adds tracks. Connectify checks if the incoming track already exists in `remoteStreamsRef.current[remoteSocketId]`, attaches it to the stream, and upserts the participant state.
- `createOffer()` / `createAnswer()` : Generates SDP session descriptions describing codecs, media formats, and connection parameters.
- `setLocalDescription()` / `setRemoteDescription()` : Configures the local and remote endpoints of the peer connection.
- `onicecandidate` : Emits ICE candidates over Socket.IO (`socket.emit('signal', remoteSocketId, { ice: event.candidate })`).
- `iceCandidateQueuesRef` : Critical race condition handler. If remote ICE candidates arrive before `setRemoteDescription()` has executed, candidates are buffered and flushed once SDP is set.
- `sender.replaceTrack()` : Used in Screen Sharing. Replaces the camera track with the display track in-place across all existing peer connections without renegotiating SDP!

STEP 8: Socket.IO Event Map

Event Name	Direction	Payload	Trigger / Action
<code>join-call</code>	Client → Server	<code>{ roomId, name, userId, videoEnabled, audioEnabled }</code>	User clicks "Join Meeting Now". Server registers user and emits <code>user-joined</code> .
<code>user-joined</code>	Server → Clients	<code>(newSocketId, allSocketIds, participantList)</code>	Existing peers trigger <code>pc.createOffer()</code> to initiate connection with the newcomer.
<code>signal</code>	Bidirectional	<code>(toSocketId, { sdp ice })</code>	Relays WebRTC session offers, answers, and ICE candidates between peers.
<code>chat-message</code>	Bidirectional	<code>(msgId, message, sender, senderName, socketId, timestamp)</code>	Broadcasts in-call chat message to room participants.
<code>video-state</code>	Bidirectional	<code>(remoteSocketId, enabled)</code>	Notifies peers when a user enables/disables their camera.
<code>audio-state</code>	Bidirectional	<code>(remoteSocketId, enabled)</code>	Notifies peers when a user mutes/unmutes their microphone.
<code>reaction</code>	Bidirectional	<code>(emoji, fromSocketId)</code>	Triggers a floating emoji reaction above the sender's video tile.
<code>user-left</code>	Server → Clients	<code>leftSocketId</code>	Triggers <code>removeParticipant()</code> , closes peer connection, and frees resources.

STEP 9: Chat System Analysis

The Connectify chat system is built directly on Socket.IO room messaging:

1. **Input & Dispatch:** User types message into `TextField` in `VideoMeet.jsx`. Pressing Enter invokes `handleSendMessage()`.
2. **Socket Dispatch:** `socketRef.current.emit('chat-message', text, displayName)`.
3. **Server Buffering & Deduplication:** In `socketManager.js`, the server assigns a cryptographically unique ID: `msgId = `${socket.id}-${Date.now()}-${Math.random().toString(36).slice(2, 7)}``.
4. **Room Storage:** Pushed to `messages[roomId]` buffer (capped at 100 entries).
5. **Broadcast:** Emitted to all sockets in the room, including the sender.
6. **Client De-duplication:** `seenMsgIds.current.has(msgId)` ref check guarantees zero duplicate messages rendered on screen.
7. **History Replay:** When a new participant joins an existing call, the server automatically replays existing `messages[roomId]` to their socket only.

STEP 10: Routing & Route Protection

Path	Component	Protection	Purpose
<code>/</code>	<code>LandingPage</code>	Public	Product showcase, marketing hero, quick join bar
<code>/auth</code>	<code>Authentication</code>	Public	Dual-tab Sign In / Create Account form
<code>/home</code>	<code>HomeComponent</code>	Protected (<code>withAuth</code> HOC)	User dashboard, instant room generator, recent meetings
<code>/history</code>	<code>History</code>	Protected (Context Token)	Full meeting log history with search & filter
<code>/:url</code>	<code>VideoMeetComponent</code>	Public / Room URL	Active video conference room & pre-meeting lobby

STEP 11: State Management Architecture

State Variable	File	Purpose	Trigger / Updated By
<code>userData</code>	<code>AuthContext.jsx</code>	Holds logged-in user profile <code>{ name, username, _id }</code>	<code>handleLogin</code> , token decode on mount
<code>inMeeting</code>	<code>VideoMeet.jsx</code>	Toggles between Pre-call Lobby and Active Meeting Stage	<code>handleJoinMeeting()</code> click
<code>localStream</code>	<code>VideoMeet.jsx</code>	Holds current user's active camera/mic <code>MediaStream</code>	<code>getUserMedia()</code> resolution on mount
<code>participants</code>	<code>VideoMeet.jsx</code>	Array of remote peers <code>[{ socketId, name, stream, ... }]</code>	<code>upsertParticipant</code> / <code>removeParticipant</code>
<code>messages</code>	<code>VideoMeet.jsx</code>	List of meeting chat messages	Socket <code>chat-message</code> event listener
<code>videoEnabled</code>	<code>VideoMeet.jsx</code>	Tracks whether local camera is broadcasting	<code>handleToggleVideo()</code> toolbar button
<code>audioEnabled</code>	<code>VideoMeet.jsx</code>	Tracks whether local microphone is unmuted	<code>handleToggleAudio()</code> toolbar button
<code>screenSharing</code>	<code>VideoMeet.jsx</code>	Tracks whether user is sharing display	<code>handleScreenShare()</code> toolbar button
<code>meetingTheme</code>	<code>VideoMeet.jsx</code>	Controls active background appearance	Settings theme selection

STEP 12: Styling & Design System

Connectify uses a sophisticated dark SaaS aesthetic built with:

- **CSS Modules:** `videoComponent.module.css` and `landing.module.css` provide scoped, collision-free styles.
- **Mobile-First Viewport:** Connectify utilizes modern `100dvh` (dynamic viewport height) units in `videoComponent.module.css` to prevent mobile browser address bars from breaking the video grid layout.
- **Adaptive CSS Grid:** Classes `grid1`, `grid2`, `grid3`, `grid4` (2×2), and `gridAdaptive` adjust video tile aspect ratios based on participant count.
- **Material UI (MUI):** Used for icons (`@mui/icons-material`) and input components (`TextField`, `Button`, `IconButton`, `Tooltip`, `Select`) styled with custom dark sx props.

STEP 13 & 14: Dependencies & Configuration

Key Backend Dependencies (`backend/package.json`)

- `express: ^5.2.1` — Next-gen HTTP routing framework featuring native promise-based error handling.
- `socket.io: ^4.8.3` — WebSocket engine for real-time WebRTC signaling and chat.
- `mongoose: ^9.7.3` — MongoDB object modeling library.
- `bcrypt: ^6.0.0` — Cryptographic password salting and hashing.
- `jsonwebtoken: ^9.0.3` — Signs and verifies HMAC-SHA256 JWT tokens.
- `cors: ^2.8.6` — Enables Cross-Origin Resource Sharing for frontend client calls.
- `dotenv: ^17.4.2` — Loads environment configuration from `.env`.

Key Frontend Dependencies (`frontend/package.json`)

- `react: ^19.2.7` & `react-dom: ^19.2.7` — Modern React 19 UI library.
- `react-router-dom: ^7.18.1` — Declarative client-side routing.
- `socket.io-client: ^4.8.3` — Client WebSocket library connecting to the signaling backend.
- `axios: ^1.18.1` — Promise-based HTTP client with request interceptors.
- `@mui/material` & `@mui/icons-material: ^9.2.0` — UI components and icons.

STEP 15: Critical Feature Data-Flow Diagrams

Feature: Start Instant Meeting

```
User in Dashboard (Home.jsx)
├── Clicks "Start Instant Meeting"
├── crypto.randomUUID().replace(/-/g, '').slice(0, 10) → generates 10-char room code (e.g. 'k9x2m4a1b0')
├── AuthContext.addToUserHistory(code) → POST /api/v1/users/add_to_activity → Saved to MongoDB
├── navigator.clipboard.writeText(url) → Invite link copied to clipboard
└── navigate(`/${code}`) → VideoMeetComponent renders Lobby view
```

Feature: Screen Sharing Track Replacement

User clicks "Share Screen"

└─ navigator.mediaDevices.getDisplayMedia({ video: true }) → Acquires screenStream

└─ screenTrack = screenStream.getVideoTracks()[0]

└─ Loop all pc in peerConnectionsRef:

 sender = pc.getSenders().find(s ⇒ s.track.kind === 'video')

 sender.replaceTrack(screenTrack) → Remote peers seamlessly see display!

└─ screenTrack.onended = () ⇒ restoreCamera() → Reverts to camera track when user stops

STEP 16, 17, 18: Issues, Security & Performance Analysis

Identified Architectural Considerations

- **Mesh Scaling Limitation:** Full-mesh WebRTC requires $N-1$ upstream and downstream connections per user. In a 6-person call, each client sends 5 video tracks and receives 5 video tracks. For > 6-8 participants, an SFU (Selective Forwarding Unit) like Mediasoup or LiveKit is recommended.
- **Room In-Memory State:** Active meeting connections and chat history live in Node.js process memory (`connections` , `messages`). In a multi-server scaled environment, a Redis Pub/Sub adapter is needed.

Security Analysis

- **Password Storage:** Strong security using `bcrypt` with 10 salt rounds.
- **JWT Authentication:** Stateless JWT verification with 7-day expiration.
- **Input Sanitization:** Custom `sanitizeName()` and regex validation on usernames prevent script injection.
- **CORS:** Currently configured with wildcard origin (`origin: "*"`); recommended to restrict to frontend origin in production.

Performance Optimizations Present

- **Isolated Memoized Subcomponents:** `MeetingTimer` , `LocalVideo` , and `RemoteVideo` use `React.memo` . Timer ticks or chat messages never cause video elements to re-render.
- **Track Mutation over Stream Recreation:** Muting video/audio modifies `track.enabled = false` instead of destroying and re-requesting `getUserMedia()` .
- **Zero-Renegotiation Screen Sharing:** Utilizes WebRTC `RTCRtpSender.replaceTrack()` avoiding costly SDP renegotiations.

STEP 19 & 20: Interview Questions & 2–Minute Pitch

Top Technical Interview Questions for Connectify

1. **Q: How does Connectify establish video calls without routing video through your Node.js server?**

A: Connectify uses WebRTC for direct Peer-to-Peer media transport. The Node.js server only acts as a signaling broker via Socket.IO to exchange SDP offers, answers, and ICE candidates. Once the DTLS/SRTP handshake finishes, audio and video stream directly between browsers.

2. **Q: How do you handle screen sharing without dropping or renegotiating peer connections?**

A: By calling `RTCTrpSender.replaceTrack(screenTrack)` on existing peer connections. This swaps the RTP video stream source seamlessly at the transport layer without triggering an SDP renegotiation offer/answer cycle.

3. **Q: How do you prevent React re-renders from glitching video playback?**

A: Peer connections and media streams are stored in React `useRef`s rather than state. Video tiles (`RemoteVideo`, `LocalVideo`) and `MeetingTimer` are wrapped in `React.memo`, isolating UI updates from video rendering.

The 2–Minute Interview Elevator Pitch

"**Connectify** is a real-time peer-to-peer video conferencing platform designed to deliver ultra-low latency collaboration without downloads or plugins.

I built the frontend in **React 19** using CSS Modules, Material UI, and React Router v7, and the backend using **Node.js**, **Express 5**, and **Socket.IO** connected to **MongoDB**.

The core audio/video engine uses **WebRTC**. While user authentication and meeting history are managed via REST APIs and MongoDB, call signaling occurs over Socket.IO rooms. When a user joins a room, existing participants generate SDP offers, receive answers, and discover network candidates via Google's STUN servers. Media streams then flow directly peer-to-peer with zero server bandwidth overhead.

Key technical challenges I solved included eliminating duplicate chat renders using unique message hashing and ref sets, managing race conditions between incoming ICE candidates and SDP descriptions using an ICE buffer queue, and enabling seamless screen sharing via `RTCTrpSender.replaceTrack()` without renegotiating peer connections."